

ENGR 102 Summer Bridge

Day 10 Instructor Key

Cumulative Recap Before Repetition

20 points • approximately 20-25 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace arithmetic and Boolean precedence.

4 points

```
x = 17
y = 5
result = (x // y) ** 2 + x % y
ok = result >= 11 and not (x % 2 == 0)
print(result, ok)
```

Show quotient, remainder, exponentiation, and both Boolean components before giving exact output.

Answer and explanation

17 // 5 is 3 and 17 % 5 is 2, so result is 3 ** 2 + 2 = 11. result >= 11 is True and [not](#) (17 % 2 == 0) is [not](#) False, also True. Exact output: 11 True.

Question 2. Trace an ordered decision with a missing-work condition.

4 points

```
score = 88
missing = True

if score >= 85 and not missing:
    message = "distinction"
elif score >= 70 or not missing:
    message = "pass"
else:
    message = "incomplete"

print(message)
```

Name the first matching branch and explain why the higher label is not selected.

Answer and explanation

The first condition is False because [not](#) missing is False. The elif condition is True because score >= 70 is True, so exact output is pass. Branch order cannot make a branch execute when its complete condition is False.

Question 3. Repair a type mismatch before comparison.

4 points

```
raw = "12"
limit = 10
print(raw >= limit)
```

Name the actual error, explain its cause, and write a corrected comparison that evaluates numerically.

Answer and explanation

Python raises TypeError because >= cannot order a string and an integer. Correct with value = int(raw) followed by print(value >= limit), which prints True.

Question 4. Program construction**8 points**

Write one complete Python program that satisfies every requirement.

- Read temperature as a float and calibrated as text, where exactly yes means calibrated.
- Print invalid when temperature is below -20 or above 60.
- Otherwise print ready only when calibrated is yes and temperature is from 15 through 35 inclusive; print hold for every other valid case.
- Use one if/elif/else chain and print exactly one word.

Answer and explanation

```
temperature = float(input())
calibrated = input()

if temperature < -20 or temperature > 60:
    print("invalid")
elif calibrated == "yes" and 15 <= temperature <= 35:
    print("ready")
else:
    print("hold")
```

The invalid range is checked first. The ready branch then combines calibration with the inclusive operating range; the final else collects every valid but not-ready case.

Protective tests include -20/yes -> hold, 15/yes -> ready, 35/yes -> ready, 36/yes -> hold, and 61/yes -> invalid.

Scoring guidance

2 input/conversion, 2 invalid boundary, 2 combined ready condition, 1 exclusive branch structure, 1 exact output/tests.